

Stock Return Prediction with Random Forests

Complete Guide: Theory, Pitfalls, and the Right Pipeline

April 21, 2026

Executive summary. For stock return prediction on a fixed universe like the S&P 500, the central discipline is *temporal*, not cross-sectional. Train on all stocks across historical dates, test on all stocks across future dates, use a standard random forest with default bootstrap, handle sector imbalance through feature engineering (sector-relative ranks) and portfolio construction (sector-neutral ranking). Cluster bootstrap by sector is a tool that exists but solves a cross-sector-generalization problem you almost certainly don't have. This document explains the theory underpinning these choices, the pitfalls that motivate them, and the concrete pipeline that ties it all together.

Contents

1	Bootstrap: the relevant mechanics	2
2	The two independence violations in stock data	3
2.1	Cross-sectional correlation (across stocks)	3
2.2	Temporal correlation (across time)	3
3	How the data is structured	3
3.1	The panel format	3
3.2	What the model sees vs. what you track	4
4	What the model learns vs. what it doesn't	4
4.1	The random forest does not see evolution	4
4.2	The apparent contradiction and its resolution	4
4.3	If you want the model to see sequences	5
5	The train/test split: the one thing you must get right	5
5.1	The rule	5
5.2	Walk-forward validation	5
5.3	Why training on 2024 and testing on 2023 is wrong	5
5.4	Purging and embargoing	5
6	Sector over-representation: the biggest practical issue	6
6.1	The S&P 500 is not balanced	6
6.2	Two different kinds of imbalance	6
7	Does the imbalance matter? Depends on use case	6
7.1	Scenario A — Universal stock picker	6
7.2	Scenario B — Cap-weighted tilted index	6

7.3	Scenario C — Long/short top- N strategy	7
8	Fixes for sector imbalance	7
8.1	Fix 1: Sector-relative features (usually the biggest win)	7
8.2	Fix 2: Sector-relative targets	7
8.3	Fix 3: Sector-neutral portfolio construction	8
8.4	Fix 4: Inverse-size sample weights	8
8.5	Fix 5: Stratified bootstrap	8
8.6	Fix 6: Separate models per sector	8
8.7	Fix 7: Expand the universe or history	8
9	The information ceiling	8
10	Bootstrap variants for stock data: the full menu	9
10.1	Cluster bootstrap (by sector)	9
10.2	Block bootstrap (by time)	9
10.3	Panel bootstrap (both axes)	9
11	Why cluster bootstrap is NOT the right tool for this project	10
11.1	What cluster bootstrap actually measures	10
11.2	Why that question doesn't apply to S&P 500 prediction	10
11.3	Correction to common advice	10
11.4	What bootstrap IS useful for here	10
12	The correct end-to-end pipeline	10
12.1	Pipeline	10
12.2	Minimal code sketch	11
13	Summary of pitfalls and remedies	12
14	Things to keep in mind when building your model	12
15	Final summary	13

1 Bootstrap: the relevant mechanics

Given a dataset D of size n , a **bootstrap sample** D^* is formed by drawing n indices *with replacement* from $\{1, \dots, n\}$. The probability that any given point is never drawn is $(1 - 1/n)^n \rightarrow 1/e \approx 0.368$, so:

- $\sim 63.2\%$ of points appear in each bootstrap sample (some duplicated).
- $\sim 36.8\%$ of points are never drawn — the **out-of-bag (OOB)** set.

How OOB tracking works in practice. Bootstrapping resamples *indices*, not values. Each bootstrap round records which indices appeared; the complement is OOB. Libraries like scikit-learn's `RandomForestRegressor(oob_score=True)` do this bookkeeping per tree: for each stock i , they average predictions only from trees where i was OOB, producing a nearly-unbiased generalization estimate.

Why duplicates don't bias the model. A point appearing k times in training is mathematically identical to the same point with weight k in the loss. Across many bootstrap samples, every point has the same *expected* weight, so no systematic bias toward particular points. The duplication perturbation is precisely what creates tree diversity in bagging.

What the bootstrap assumes. Draws are i.i.d. For stock data this assumption is violated along two axes simultaneously — see Section 2.

2 The two independence violations in stock data

2.1 Cross-sectional correlation (across stocks)

At any given moment, stocks within the same sector share underlying drivers:

- Tech stocks co-move with chip demand, rate expectations, AI sentiment.
- Energy stocks co-move with oil prices.
- Banks co-move with yield curves and credit spreads.

Treating each stock as an independent draw dramatically understates the true correlation structure — AAPL and MSFT on any given day are close to statistical near-duplicates.

2.2 Temporal correlation (across time)

Stock returns exhibit three distinct forms of temporal dependence, each relevant to different bootstrap concerns:

- **Volatility clustering.** Large moves come in bursts (2008, March 2020). Periods of calm and periods of stress are persistent states, not memoryless draws.
- **Autocorrelation.** Small but real serial dependence in returns themselves, and strong dependence in squared returns (volatility persistence).
- **Regime shifts.** Bull/bear markets, rate cycles, crisis regimes, post-crisis recoveries. The full distribution of returns is non-stationary.

Consequence. A naive bootstrap across all (stock, day) cells shatters both structures. Train and OOB end up containing near-duplicates of each other, and OOB error becomes meaningless — the model appears to generalize because it is silently memorizing correlated patterns.

3 How the data is structured

3.1 The panel format

Your data is a 2D grid of (stock, time). You unfold it into a long-format table where each row is one (stock, time) snapshot:

Row	Stock	Sector	Date	P/E	Momentum	Volatility	Next-period return
1	AAPL	Tech	2023-01	28.5	0.12	0.18	0.03
2	AAPL	Tech	2023-02	29.1	0.15	0.17	0.02
3	MSFT	Tech	2023-01	32.1	0.08	0.15	0.02
4	XOM	Energy	2023-01	12.3	−0.05	0.22	−0.01
5	JPM	Finance	2023-01	10.2	0.04	0.19	0.01
⋮							

3.2 What the model sees vs. what you track

Column	Goes to model?	Purpose
Feature columns (P/E, Momentum, ...)	Yes	The model's input X
Target column (next-period return)	Yes	The model's label y
Stock identifier	No	Metadata; feature computation, diagnostics
Date	No	Metadata; train/test split
Sector	No	Metadata; normalization, portfolio construction

What the model actually learns. A timeless, cross-sectional mapping: “given these feature values, expect this return.” The model has no concept of which stock, which date, or which sector a row came from. It is a pure pattern matcher on feature vectors.

4 What the model learns vs. what it doesn't

4.1 The random forest does not see evolution

Each row is a snapshot. The model processes rows independently; it does not know rows 1 and 2 are AAPL at two different dates, nor that rows 1 and 3 are both tech stocks in January 2023. All history present in the features is pre-digested into scalar summaries before the model sees it.

Information	In features?	How it enters the model
Current valuation	Yes	P/E column
Recent trend	Yes (compressed)	Momentum column
Recent volatility	Yes (compressed)	Volatility column
Raw price sequence	No	Never reaches the model
Stock identity	No	Metadata only
Calendar date	No	Metadata only
Market regime	Only if added as feature	e.g., VIX level, yield curve

4.2 The apparent contradiction and its resolution

The puzzle. If the model is time-blind and processes rows independently, why does it matter how we split train and test?

The answer: the model's time-blindness is precisely what makes it vulnerable to leakage.

Two rows from adjacent dates (e.g., March 15 and March 16 for the same stock) have nearly identical feature values AND nearly identical labels. If March 15 is in training and March 16 is in test, the model pattern-matches the test features to their training neighbor and retrieves the training label. Test error looks great. But the model has memorized a neighbor, not learned to forecast. In real deployment, you don't have March 15 labeled when predicting March 16.

The correct framing. Correlation between rows doesn't break the *model*. It breaks the *test score*. The test score stops being an honest estimate of deployment performance when correlated rows straddle the train/test boundary. The model's simplicity is what makes it useful for generalization *and* what makes it easy to fool by near-duplicates.

4.3 If you want the model to see sequences

Three options, in increasing order of complexity:

1. **Lag features.** Add columns $x_{t-1}, x_{t-2}, \dots$ for each original feature. Still fixed-width, still random-forest-compatible. Each row now contains a short trajectory, and the RF can learn patterns like “rising momentum $\rightarrow$ positive next return.”
2. **Dynamics features.** Engineered columns like “change in momentum over 3 months,” “volatility acceleration,” “distance from 52-week high.”
3. **Sequence models.** LSTMs or Transformers genuinely process sequences. Input becomes a sequence per (stock, prediction date). Needed when temporal pattern itself carries signal. Often underperforms well-engineered RFs on noisy financial data, because added capacity mostly fits noise.

5 The train/test split: the one thing you must get right

5.1 The rule

Train on the past; test on the future. Always.

Training data must contain only information that would have been available at the moment of any test-time prediction. Violating this produces a test score that measures retrofitting, not forecasting.

5.2 Walk-forward validation

Multiple evaluation folds, each preserving the temporal ordering:

Fold	Training window	Test window
1	2015–2020	2021
2	2015–2021	2022
3	2015–2022	2023
4	2015–2023	2024

Every fold includes every stock and every sector in both training and testing. The *only* difference between training and test is the date range.

5.3 Why training on 2024 and testing on 2023 is wrong

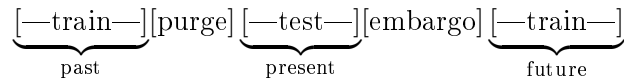
This is counterintuitive because the model is time-blind, yet the direction still matters. The reason: training data shapes the statistical distribution the model calibrates to. If that distribution contains 2024 information, the model implicitly “knows” what came after 2023 — market regimes, post-crisis resolutions, correlations that only emerged later. A real forecaster in 2023 wouldn’t have had this. Your test score would measure “how well does a model with oracle access to the future explain the past,” which is not a useful quantity.

5.4 Purging and embargoing

Two technical refinements that matter when labels span multiple days:

- **Purge.** If the target at time t covers a forward window (e.g., return from t to $t + 21$), drop any training row whose label window overlaps the test window. Otherwise the training label literally contains information about the test label.

- **Embargo.** After a test block ending at date T , exclude the next few days from the subsequent training block. Prevents serial-correlation leakage at the boundary.



6 Sector over-representation: the biggest practical issue

6.1 The S&P 500 is not balanced

Approximate breakdown (recent years):

Sector	Approx. # stocks	Approx. index weight
Technology	70–80	~30%
Financials	65	~13%
Healthcare	60	~12%
Consumer Discretionary	55	~10%
Industrials	70	~8%
Communication Services	25	~8%
Consumer Staples	35	~6%
Energy	22	~4%
Utilities	30	~3%
Real Estate	30	~2%
Materials	28	~2%

6.2 Two different kinds of imbalance

1. **Count imbalance.** Tech contributes many more training rows than utilities, so the loss function is dominated by tech patterns. A naive model is systematically more accurate on tech than on small sectors.
2. **Market-cap imbalance.** Even within a sector, a few names (Apple, Microsoft) dwarf the others in portfolio weight. In real-money terms, a 1% error on Apple costs vastly more than a 1% error on a small utility.

These pull in opposite directions, which is part of what makes handling imbalance nuanced.

7 Does the imbalance matter? Depends on use case

7.1 Scenario A — Universal stock picker

Goal. Predict any stock well. Pick the single best stock in the market each month.

Impact of imbalance. Significant. The model is better at tech than utilities, so utility predictions are noisier. When it ranks all 500 stocks by predicted return, low-confidence noise on utilities can propel a spurious pick to the top.

Verdict. Imbalance hurts. Needs active fixes.

7.2 Scenario B — Cap-weighted tilted index

Goal. Overweight predicted winners within a cap-weighted S&P 500 tracker.

Impact of imbalance. Approximately aligned. Tech dominates your portfolio weight (30%) and dominates your training signal (and hence model accuracy). A 1% error on Apple matters more to you than a 1% error on a small utility — exactly matching where the model is more accurate.

Verdict. Imbalance is not a problem. Possibly no fix needed.

7.3 Scenario C — Long/short top- N strategy

Goal. Long the top 50 predicted winners, short the bottom 50 predicted losers, dollar-neutral.

Impact of imbalance. Severe. Three distinct failure modes:

1. **Sector concentration in picks.** Model is more *confident* on tech, so top/bottom 50 ends up dominated by tech. Portfolio becomes a hidden long-tech / short-tech bet instead of a cross-sectional spread.
2. **Missed opportunities in small sectors.** A utility about to rally 15% may not make the top 50, because the model can't reliably identify utility outliers.
3. **Overconfident extremes.** When the model does put a utility at the top, it's often noise — small-sector picks are disproportionately wrong.

Verdict. Imbalance hurts most of all here. This is the scenario where fixes matter most.

Strategy	Imbalance hurts?	Why
Pick one best stock globally	Yes, significantly	Model noisier on small sectors
Cap-weighted portfolio tilt	No, approximately aligned	Model focus matches portfolio focus
Long/short top/bottom N globally	Yes, severely	Tail picks concentrate in tech
Long/short top/bottom N per sector	No, handled by construction	Balance enforced at portfolio stage

8 Fixes for sector imbalance

Seven techniques, ranked by practical impact:

8.1 Fix 1: Sector-relative features (usually the biggest win)

Replace raw features with within-sector ranks or z-scores:

$$\text{momentum}_i^{\text{rel}} = \text{rank within (sector, date)}(\text{momentum}_i)$$

Now the model learns one general pattern — “high rank within its sector outperforms” — pooling information from all 500 stocks. A utility ranked 0.9 is in the top 10% of utilities; a tech stock ranked 0.9 is in the top 10% of tech. Same scale, cross-sector comparable. This often removes the imbalance problem by itself.

8.2 Fix 2: Sector-relative targets

Predict *excess return over sector mean* instead of raw return:

$$y_i^{\text{rel}} = r_i - \bar{r}_{\text{sector}(i),t}$$

The learning objective becomes sector-neutral by construction. Combined with Fix 1, this is the standard quant-finance setup.

8.3 Fix 3: Sector-neutral portfolio construction

Even with a biased model, force balance at the trading stage: long/short the top/bottom within each sector, not globally.

Longs: top 5 per sector $\times$ 11 sectors = 55 stocks

Shorts: bottom 5 per sector $\times$ 11 sectors = 55 stocks

Prevents sector concentration regardless of model bias.

8.4 Fix 4: Inverse-size sample weights

Weight each row so each sector contributes equally to the loss:

$$w_{\text{tech}} = \frac{1}{140}, \quad w_{\text{utility}} = \frac{1}{15}, \quad \dots$$

Supported directly in scikit-learn via `sample_weight`. Useful supplement. **Caveat:** reallocates the model's *attention* but does not create new information. See the information ceiling below.

8.5 Fix 5: Stratified bootstrap

When drawing each bootstrap sample, draw a fixed number of stocks *per sector* rather than uniformly from all 500. Each tree sees balanced sector representation. Requires custom RF construction; rarely the biggest lever compared to Fix 1.

8.6 Fix 6: Separate models per sector

Train one model per sector. Cleanly removes imbalance but loses cross-sector learning. Useful when sectors behave very differently (commodities vs. banks) or when you suspect a single model can't capture the heterogeneity. Uses your data less efficiently.

8.7 Fix 7: Expand the universe or history

If a small sector has genuinely insufficient data, add mid-cap stocks from that sector (Russell 3000) or extend the time history. No weighting scheme creates data that doesn't exist.

The practical recipe. Fix 1 + Fix 2 + Fix 3 (sector-relative features, sector-relative targets, sector-neutral portfolio construction). This three-layer stack handles imbalance at the feature level, the learning objective level, and the deployment level. It is the de-facto standard in quantitative equity and is vastly more effective than any single sampling-based fix.

9 The information ceiling

Critical distinction. Reweighting or stratification tells the model *to try harder* on a small subgroup. It does not tell the model *how* to do better when the data itself is too sparse.

Two failure modes appear when you over-weight a small subgroup:

- **Overfitting.** Heavy weights on 15 utility stocks makes the model memorize their specific idiosyncrasies. Training error on utilities looks great; test error doesn't follow.

- **High prediction variance.** Different bootstrap trees see different tiny subsets of the 15 utilities. Predictions swing widely. Measurable as the standard deviation of predictions across trees, per stock — utilities will show markedly wider spread than tech.

Practical diagnostic. Compute per-stock standard deviation of predictions across bagged trees. If small sectors show much higher variance than large sectors even after weighting, you have hit the information ceiling. No clever weighting fixes this — you need more data (Fix 7) or a different formulation (Fix 1, which pools information across sectors).

10 Bootstrap variants for stock data: the full menu

This section covers what the various bootstrap variants do, for completeness, before the next section explains why most of them aren't the right tool for S&P 500 return prediction.

10.1 Cluster bootstrap (by sector)

Treat each **sector** as the atomic sampling unit. When a sector is drawn, all its stocks come along as a bundle — all-or-nothing per sector.

	Standard bootstrap	Cluster bootstrap
Sampling unit	Individual stock	Entire sector
Tech in OOB	Partial (some in, some out)	All-or-nothing
Leakage risk	High (sector twins leak)	Low

What it measures. OOB error reflects how well the model would perform on an entire sector it has never seen. Useful for cross-sector generalization questions.

10.2 Block bootstrap (by time)

Sample contiguous blocks of time rather than individual days. Variants:

- **Moving block bootstrap.** Fixed-length blocks starting at any day. Resample enough blocks to match original length.
- **Stationary bootstrap** (Politis–Romano). Block lengths drawn from a geometric distribution; nicer theoretical properties.
- **Circular block bootstrap.** Treats the series as a loop so endpoints aren't underrepresented.

Typical block length for daily equity data: $L \in [20, 60]$ days — long enough to preserve volatility clusters, short enough to allow many independent positions.

What it measures. Used for computing confidence intervals on time-series-dependent statistics (Sharpe ratio, drawdown, etc.) while respecting volatility clustering.

10.3 Panel bootstrap (both axes)

Sample (sector, time-block) pairs. Each draw brings in all stocks of the sector for the entire block:

Draw	Block	Rows added
1	(Tech, Jan–Mar 2020)	70 tech stocks $\times$ 60 days = 4200 rows
2	(Energy, Jul–Sep 2022)	25 energy stocks $\times$ 60 days = 1500 rows
3	(Tech, Jan–Mar 2020)	Duplicate — same rows
$\vdots$		

OOB = all (sector, time-block) cells never drawn. Preserves both correlation axes simultaneously.

11 Why cluster bootstrap is NOT the right tool for this project

Despite the theoretical elegance of cluster/panel bootstraps, they answer a question you are not asking.

11.1 What cluster bootstrap actually measures

“How well does my model generalize to sectors it has never seen?”

11.2 Why that question doesn’t apply to S&P 500 prediction

- Your universe always contains all 11 sectors.
- At deployment time, you always face familiar sectors.
- Holding out whole sectors during training throws away useful information without reflecting any realistic deployment scenario.

11.3 Correction to common advice

“The data is clustered, therefore use cluster bootstrap” is a textbook reflex that doesn’t apply here. Cluster bootstrap is correct when you intend to deploy on genuinely new clusters (new asset classes, new geographies, new markets). For stock prediction within a fixed universe where all sectors are always present, cluster bootstrap solves a problem you don’t have.

11.4 What bootstrap IS useful for here

- **The random forest’s internal bootstrap** for variance reduction via bagging — no action required, just use the default.
- **Block bootstrap over time** for computing confidence intervals on performance metrics (Sharpe, hit rate, IC) from the test-period returns. Preserves volatility clustering in the uncertainty estimate.

Bootstrap variant	Use for S&P 500?	Purpose
Internal RF bootstrap (default)	Yes	Ensemble variance reduction; auto
Block bootstrap over time	Yes, for metrics	Error bars on Sharpe, IC, etc.
Cluster bootstrap by sector	No	Solves wrong problem for this use
Panel bootstrap (sector $\times$ time)	No	Same reason as cluster bootstrap
Standard bootstrap on individual rows	Internal only, not for evaluation	Shatters both correlation structure as an evaluation tool

12 The correct end-to-end pipeline

12.1 Pipeline

1. **Build panel data.** One row per (stock, date). Features computed using only information available at that date (no lookahead).

2. **Cross-sectional normalization.** Within each (date, sector), rank each feature. Output: sector-relative ranks in $[0, 1]$.
3. **Target transformation.** Convert raw returns to sector-relative returns (subtract sector-date mean) for a sector-neutral learning objective.
4. **Walk-forward split.** Train on historical dates; test on future dates. Apply purging if label windows overlap. Optionally use expanding or rolling windows.
5. **Train.** Standard `RandomForestRegressor` on training rows. Default bootstrap. No sector holdouts.
6. **Predict.** Model outputs a return forecast for each (stock, date) in test.
7. **Portfolio construction.** Rank predictions within each (test-date, sector). Long the top k per sector; short the bottom k per sector.
8. **Evaluate.** Compute Sharpe, hit rate, IC on the portfolio's test-period returns. Use block bootstrap over time for confidence intervals.
9. **Diagnose.** Per-sector accuracy; per-sector prediction variance across trees; turnover; realized sector exposure of live positions.

12.2 Minimal code sketch

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor

# 1. Panel assumed built: columns stock, date, sector, pe, mom, vol, y

# 2. Cross-sectional sector-relative features
for col in ['pe', 'mom', 'vol']:
    panel[col + '_rel'] = (panel.groupby(['date', 'sector'])[col]
                           .rank(pct=True))

# 3. Sector-relative target
panel['y_rel'] = (panel.groupby(['date', 'sector'])['y']
                  .transform(lambda x: x - x.mean()))

# 4. Walk-forward split
train = panel[panel['date'] < '2023-01-01']
test = panel[panel['date'] >= '2023-01-01']

feat_cols = ['pe_rel', 'mom_rel', 'vol_rel']

# 5. Train
model = RandomForestRegressor(n_estimators=500, n_jobs=-1, random_state=0)
model.fit(train[feat_cols], train['y_rel'])

# 6. Predict
test = test.copy()
test['pred'] = model.predict(test[feat_cols])

# 7. Sector-neutral portfolio: top/bottom 5 within each (date, sector)
test['rank_desc'] = (test.groupby(['date', 'sector'])['pred']
                     .rank(method='first', ascending=False))
test['rank_asc'] = (test.groupby(['date', 'sector'])['pred']
                    .rank(method='first', ascending=True))
```

```
longs = test[test['rank_desc'] <= 5]
shorts = test[test['rank_asc'] <= 5]
```

13 Summary of pitfalls and remedies

Pitfall	Why it breaks	Remedy
Random shuffled train/test split	Future-to-past leakage; correlated neighbors across boundary	Walk-forward split
Training on later dates than test	Training distribution contains future information	Strict past-to-future ordering
Overlapping label windows across train/test	Test label literally in training data	Purge
Adjacent-day leakage at train/test boundary	Serial correlation, slow feature evolution	Embargo
Heavy tech weighting in raw training data	Model biased to tech patterns	Sector-relative features (primary) + sample weights (supplement)
Global long/short picks concentrated in tech	Model more confident on large sectors	Sector-neutral portfolio construction
Small sectors with too few stocks	Information ceiling on learnability	Expand universe; use sector-relative features to pool information
Expecting RF to “see evolution”	Each row is a snapshot	Lag features, engineered dynamics, or sequence model
Relying on internal RF OOB error as main metric	Cross-sectional correlation makes OOB optimistic	Use walk-forward test error as primary metric
Using cluster bootstrap because “data is clustered”	Wrong question for this use case	Skip it; use block bootstrap over time for metric CIs

14 Things to keep in mind when building your model

Checklist of what actually matters

1. Walk-forward split. Train strictly on the past, test strictly on the future. No exceptions, ever, for any reason.
2. Features computed using only past information relative to the row’s date. No lookahead.
3. Sector-relative features so scales are comparable across sectors.
4. Sector-relative target (if doing long/short) so the learning objective is sector-neutral.
5. Default random forest. Do not hand-roll cluster bootstrap.
6. Sector-neutral portfolio construction to prevent tech concentration at the tails.
7. Block bootstrap over time for confidence intervals on performance metrics.
8. Purge overlapping label windows between train and test. Embargo the boundary.
9. Diagnose per-sector: accuracy, prediction variance across trees, realized exposure.
10. Remember the model is time-blind and identity-blind. If regime or sequence information matters, engineer it explicitly into features.

Common mistakes to avoid

- Random train/test split that shuffles dates — destroys temporal ordering.
- Training on 2024 and testing on 2023 “because the specific rows differ” — leaks future distribution information.
- Using the RF’s internal OOB error as the main quality metric — optimistic due to cross-sectional correlation; use walk-forward test error instead.
- Reaching for cluster bootstrap because “data is clustered” without checking whether cross-sector generalization is the question you care about.
- Fighting sector imbalance only at the sampling level while leaving raw features in place — sector-relative features do more than any sampling fix.
- Expecting a random forest to learn market flow. It cannot. Regime features must be engineered in, or switch to a sequence model.
- Computing features that use future information (e.g., “rank within the full time series” when the future hasn’t happened).
- Over-weighting small sectors without checking for overfitting or high prediction variance — the information ceiling bites regardless of weighting.

15 Final summary

The three-line version of this entire document:

1. For S&P 500 return prediction, the only holdout that matters is *temporal* — train on all stocks across past dates, test on all stocks across future dates.
2. Handle sector imbalance with *sector-relative features*, *sector-relative targets*, and *sector-neutral portfolio construction* — not with custom bootstrap schemes.
3. Use a *default random forest*. The honesty of your evaluation comes from the walk-forward split, not from anything clever inside the forest.